

Arquitetura e Padrões de Software

Temas dos Trabalhos

Prof. Marcelo Arêas

Temas para 2026.1

- Arquitetura Blackboard
- Arquitetura SOA
- Arquitetura de Microsserviços
- Arquitetura Baseada em Eventos (com IoT)

OBS: Apesar do foco ser a arquitetura, todos os trabalhos devem possuir um front-end minimamente simples para permitir uma fácil execução e demonstração de uso.

Arquitetura de Memória Compartilhada: Blackboard

- **Tema: Sistema de controle e abastecimento de estoque para lojas de varejo**
- Criar um sistema para gerenciar o estoque de produtos em uma rede de loja de varejos, utilizando o estilo arquitetural Blackboard.
- Armazenar no Blackboard dados como níveis de produtos no estoque por loja, histórico de vendas, dados de fornecedores de produtos e histórico de compras de fornecedores.
- Criar dois módulos especialistas representando lojas de varejo, que devem atualizar o Blackboard a cada venda realizada.
- Criar um terceiro módulo especialista para gerenciar a reposição de estoque. Esse módulo deve sugerir ordens de compra ou transferências de produtos entre lojas com base no estoque disponível.

Arquitetura SOA

- **Tema: Plataforma de serviços para reserva de espaços em condomínio**
- Criar uma plataforma para um condomínio residencial composta por um conjunto de *web services* que podem ser consumidos por sistemas independentes. A arquitetura deve incluir serviços como:
 - Consulta de disponibilidade de churrasqueira/salão de festa em um intervalo de dias: Acessa uma base de dados (previamente preparada) para retornar o status de disponibilidade de cada churrasqueira/salão de festa nas datas do intervalo.
 - Reserva de espaço: Deve enviar nome, cpf, espaço, identificação da churrasqueira/salão de festa, data. O serviço deve fazer a devida reserva e retornar sucesso ou falha.
 - Consulta de suas reservas: Deve enviar cpf e intervalo de datas. Retorna uma lista com as reservas efetuadas pelo cpf para o intervalo.
 - Relatório das reservas (administrativo): Deve enviar um intervalo de datas. Retorna todas as reservas feitas por todos os moradores nesse intervalo.
- Os serviços devem seguir padrão de comunicação SOAP/WSDL, com arquivos no formato XML.
- Para consultar os serviços devem ser construídas duas aplicações clientes simples, que possibilite fornecer os dados de entrada e receba a resposta. As aplicações devem simular app móvel dos moradores e um sistema disponível na portaria.

Arquitetura de Microserviços

- **Tema: Sistema de gestão de clínicas**
- Criar um sistema para gestão de clínicas médicas com múltiplas especialidades (cardiologia, ortopedia, pediatria...), baseada em microserviços independentes. O sistema deve contemplar serviços como:
 - Cadastro de pacientes
 - Cadastro de médicos com especialidades
 - Agendamento de consultas
 - Cancelamento e remarcação
 - Faturamento/cobrança
- Identificar os diferentes domínios de negócios e suas funcionalidades, como CRUD de pacientes e médicos, especialidades, escala de atendimento.
- Paciente pode se cadastrar, editar, marcar consulta, remarcar consulta, solicitar cancelamento.
- Administrador pode cadastrar médicos especialistas, aprovar ou reprovar solicitação de cancelamento, emitir fatura/cobrança de consulta.
- Projete a arquitetura de forma a ter pelo menos 3 microserviços. Cada microserviço deve estar em um container Docker independente, explorando questões como escalabilidade, tolerância a falhas e comunicação entre microserviços.
- Exemplo de tolerância a falhas: serviço de agendamento fora do ar e restante do sistema funcionando. Ou serviço de faturamento fora do ar e restante do sistema funcionando.
- A comunicação pode ser feita via APIs REST, no formato JSON.

Arquitetura Baseada em Eventos (com IoT)

- **Tema: Sistema de Monitoramento Ambiental em Tempo Real**
- Criar um sistema para monitorar dados ambientais como qualidade do ar, temperatura, umidade, poluição sonora, risco de tempestade.
- Esses dados serão provenientes de sensores distribuídos que publicam os eventos constantemente. Para simplificar, simule o funcionamento dos sensores (dados fixos manuais, ou dados aleatórios).
- Cada leitura de um sensor é considerado um evento, que pode ser consumido por diferente serviços, como:
 - Alerta em tempo real de possível risco, para autoridades ou moradores;
 - Painel/mosaico de visualização dos dados em tempo real;
 - Armazenamento para análise histórica;
- O objetivo é mostrar como fluxos de dados em tempo real podem ser tratados com eficiência. E como sistemas reativos baseados em eventos contribuem com escalabilidade e a modularidade.
- A aplicação para simular os sensores (*publish*) deve ser separada da aplicação dos serviços (*subscribe*). A comunicação deve utilizar um Message Broker para os eventos (RabbitMQ, por exemplo).

Entrega

- A entrega será realizada como um trabalho no Teams
- Apenas um membro da equipe deve realizar a entrega
- Atenção para o prazo da entrega, que será definido. É possível fazer mais de uma entrega e atualizar, mas não é possível entregar após o prazo.
- O arquivo de entrega deverá estar compactado em formato **zip** ou **rar** e conter:
 - Código fonte do sistema desenvolvido
 - Documentação

Documentação

- Documento em formato **pdf** descrevendo os padrões de software que foram utilizados e porquê. Pode incluir recortes de diagramas de classes.
- Documento em formato **pdf** contendo a descrição do sistema (criar o problema e o sistema como solução) e as suas preocupações (*concerns*). Descrever quais são os principais pontos de vistas e perspectivas que endereçam essas preocupações.
- Diagrama de implantação, retratando os módulos do sistema.
- Diagramas de casos de uso, para listar funcionalidades por tipos de atores envolvidos.
- Diagramas de atividades, para detalhar as funcionalidades e suas interações com atores e módulos.
- Diagrama de estados, apenas se o sistema possuir alguma máquina de estado.
- *Atenção: Diagramas devem ser legíveis! Um diagrama que não se consegue ler o que está nele, é um diagrama não entregue.*

Critério de avaliação

- Avaliação do sistema, pelo uso correto da arquitetura e padrões de software: 5,0
- Conformidade da documentação: 3,0
- Apresentação: 2,0
 - 15 a 20 minutos, explicando o problema proposto, os *concerns*, e o sistema como solução, demonstrando todas as funcionalidades do sistema desenvolvido;
 - Todos os integrantes do grupo devem estar presente;
 - Exceção: viagens a trabalho pré-justificadas (vídeo) ou motivo de doença (atestado);
 - Ao final podem haver perguntas.

OBS: A ordem de apresentação dos grupos será sorteada.

OBS 2: Independente do dia da apresentação, a data de entrega do trabalho via Teams será igual para todos os grupos.